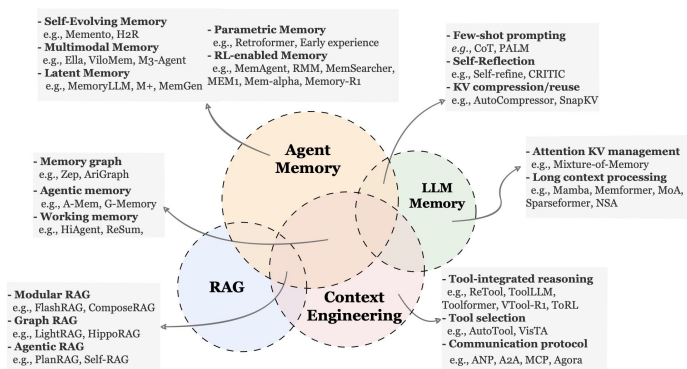


Transformers in Deep Learning

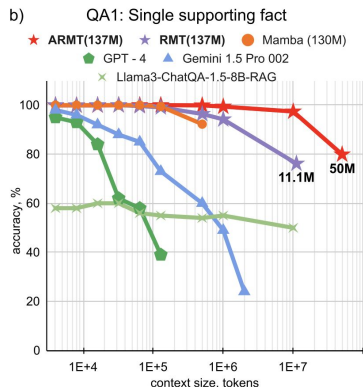
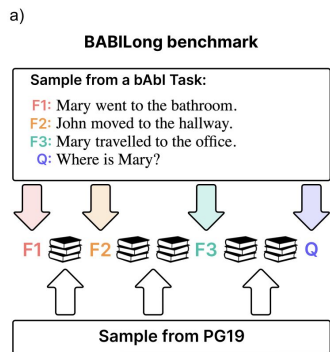
Lecture 9

Week 9



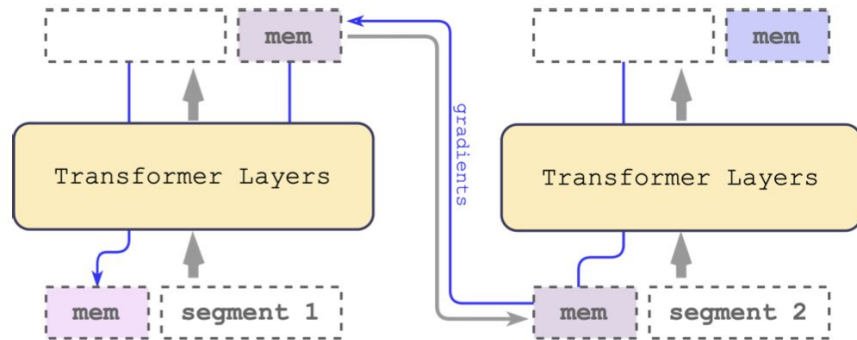
<https://arxiv.org/abs/2512.13564>

Memory Evaluation



<https://arxiv.org/abs/2406.10149>

RMT



<https://arxiv.org/abs/2207.06881>

Memory Compression

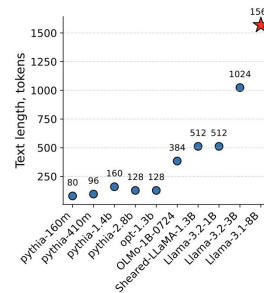
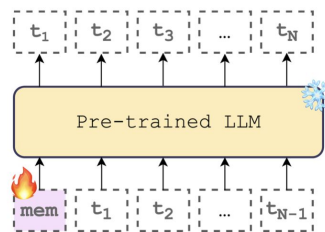
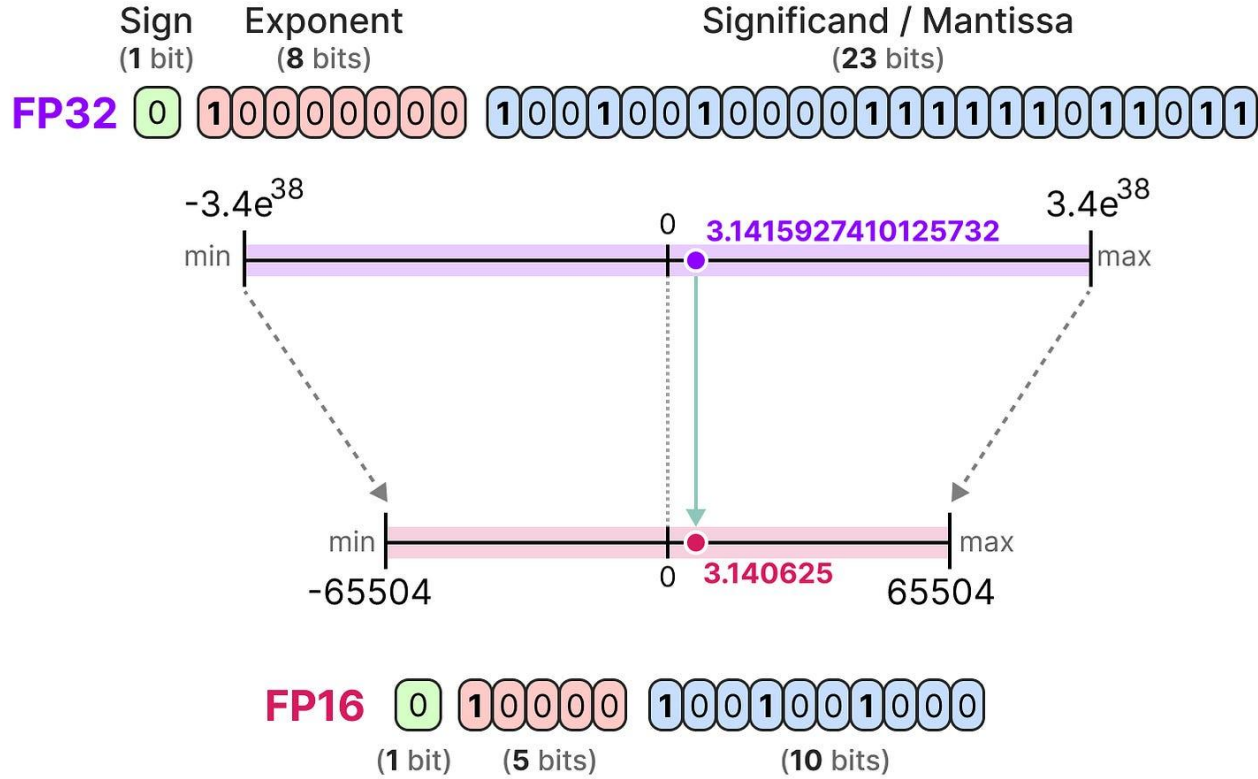


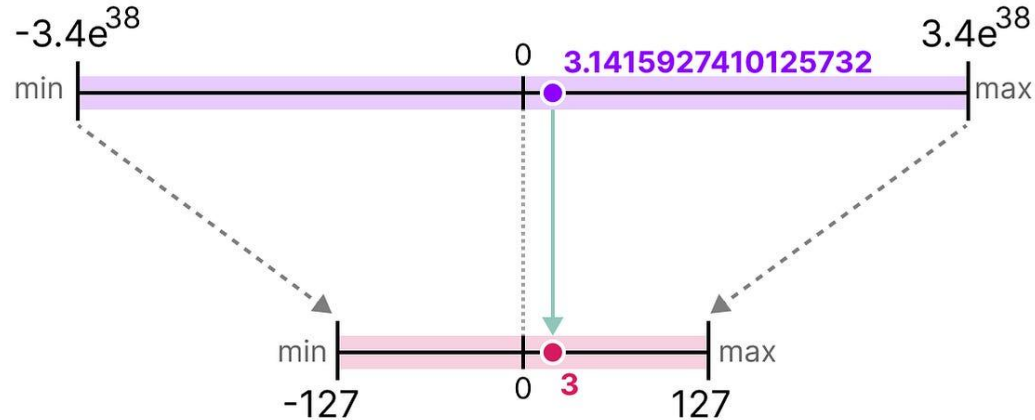
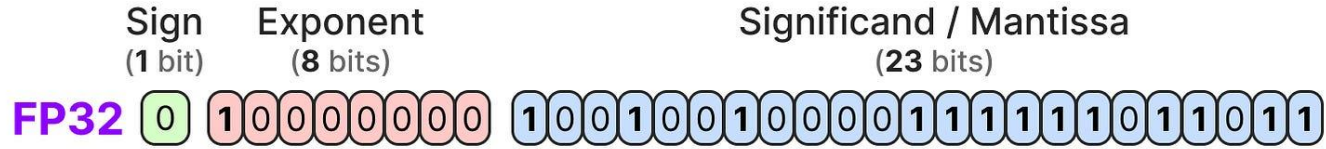
Figure 1: How many tokens fit into a single input vector? We estimate maximum number of tokens that can be decoded from a single input vector across various language models.

<https://arxiv.org/abs/2502.13063>

Lecture 8 Recap



Lecture 8 Recap



Lecture 8 Recap

$$s = \frac{2^{b-1} - 1}{\alpha} \quad (\text{scale factor})$$

$$x_{\text{quantized}} = \text{round}(s \cdot x) \quad (\text{quantization})$$

Filling in the values would then give us the following:

$$s = \frac{127}{10.8} = 11.76 \quad (\text{scale factor})$$

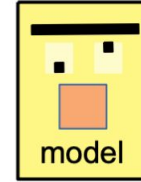
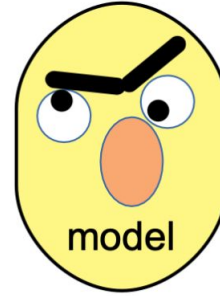
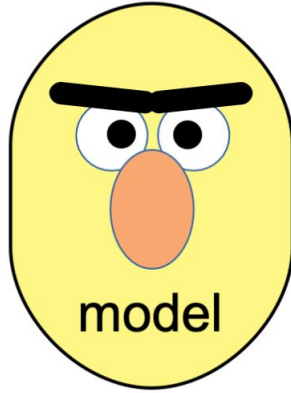
$$x_{\text{quantized}} = \text{round}(11.76 \cdot \text{[array of 8 purple squares]}) \quad (\text{quantization})$$

To retrieve the original FP32 values, we can use the previously calculated *scaling factor* (s) to *dequantize* the quantized values.

$$x_{\text{dequantized}} = \frac{\text{[array of 8 pink squares]}}{s} \quad (\text{dequantize})$$

- b is the number of bytes that we want to quantize to (8),
- α is the *_highest* absolute value,

Lecture 8 Recap



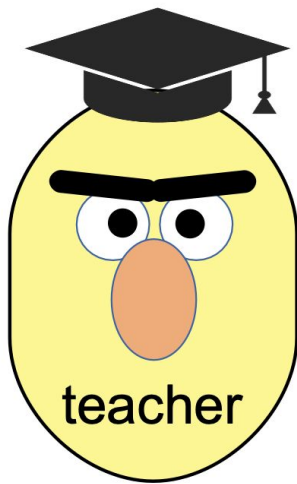
FP16



INT8



Lecture 8 Recap

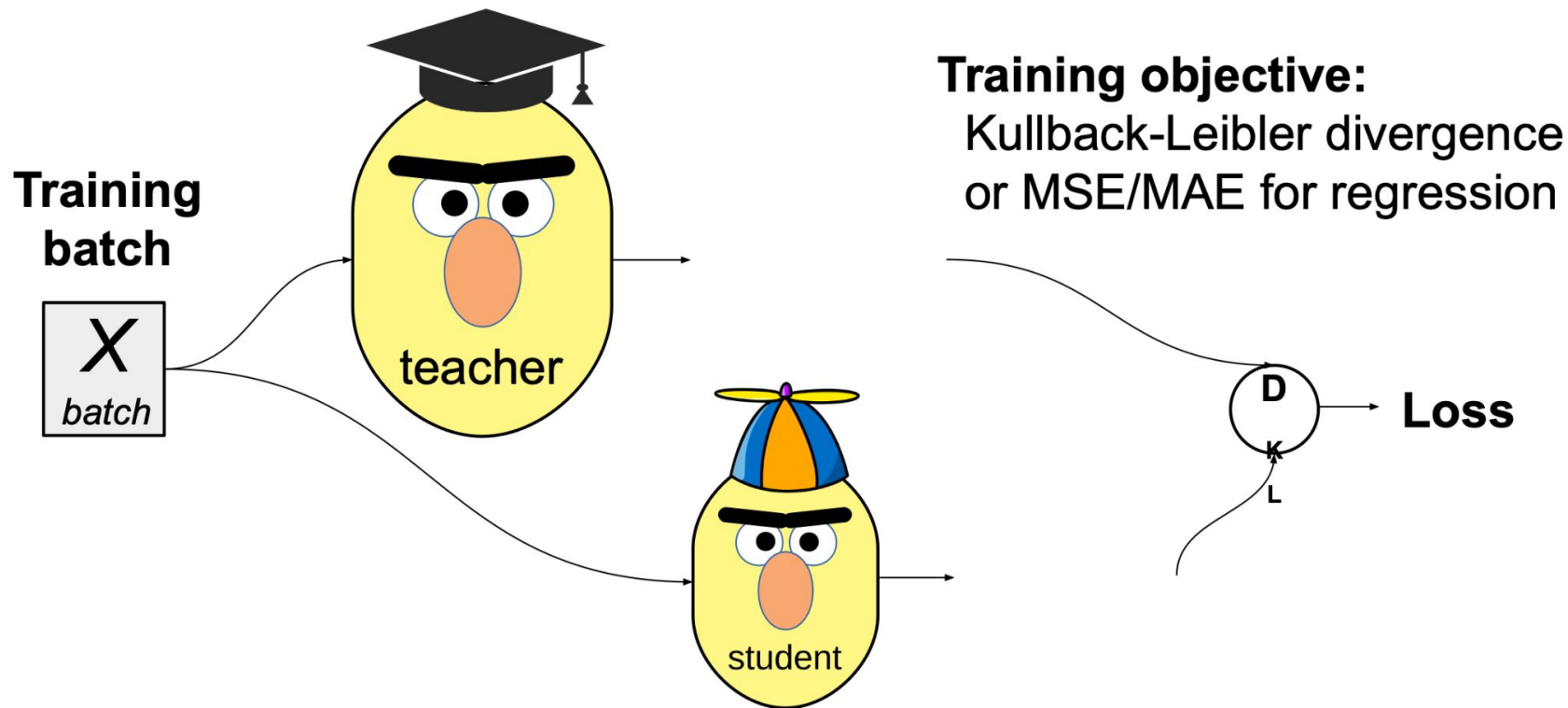


First, get the best
performing model
regardless of size

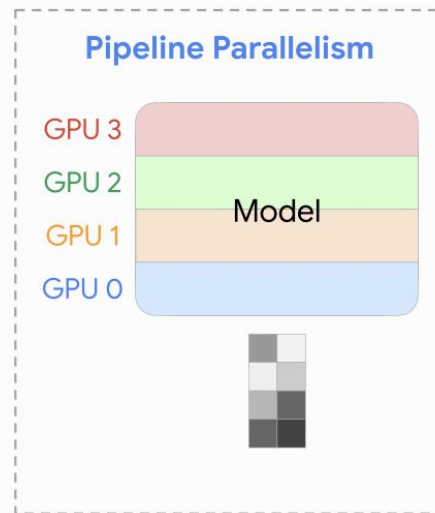
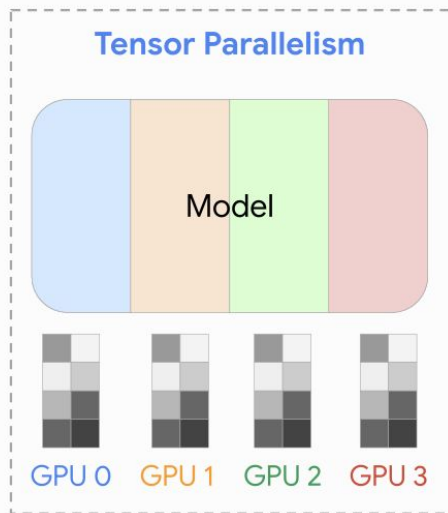
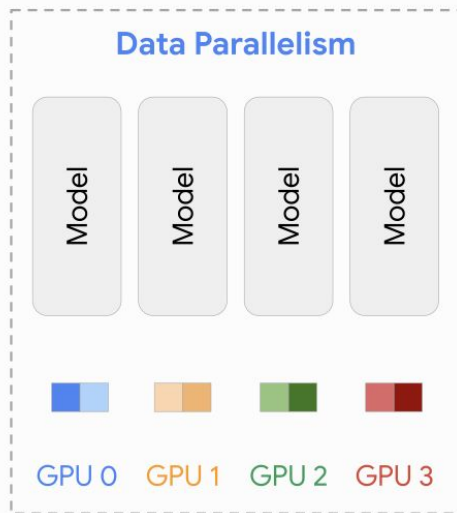


Then, train a more
compact model to
approximate it!

Lecture 8 Recap



Distributed training



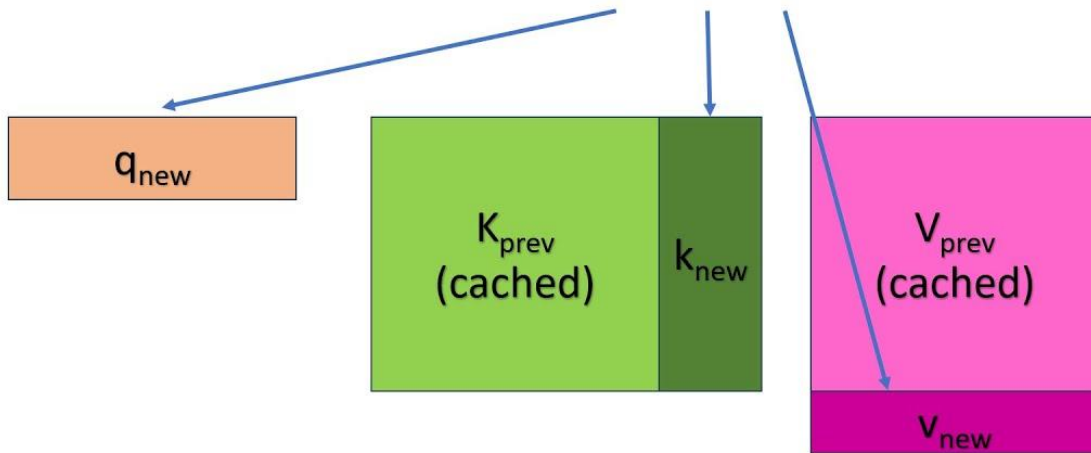
Efficient Attention

Attention Complexity

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Recurrent	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

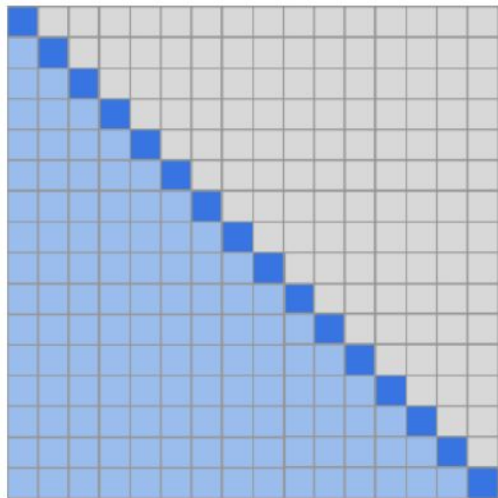
KV-cache

KV Cache

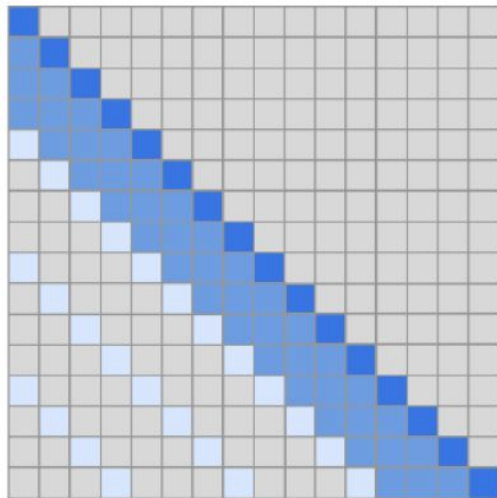


$x_1, x_2, \dots, x_n, x_{n+1}, x_{n+2} \rightarrow x_{n+3}$

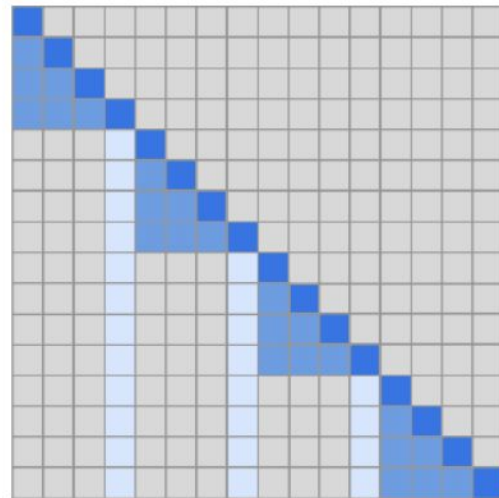
Attention matrix



(a) Transformer

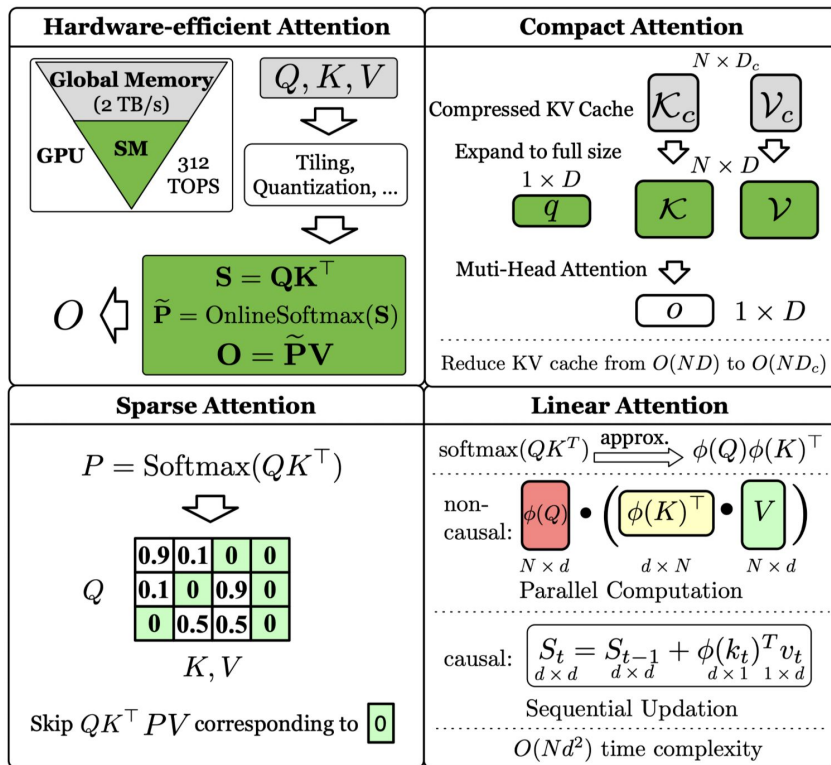


(b) Sparse Transformer (strided)

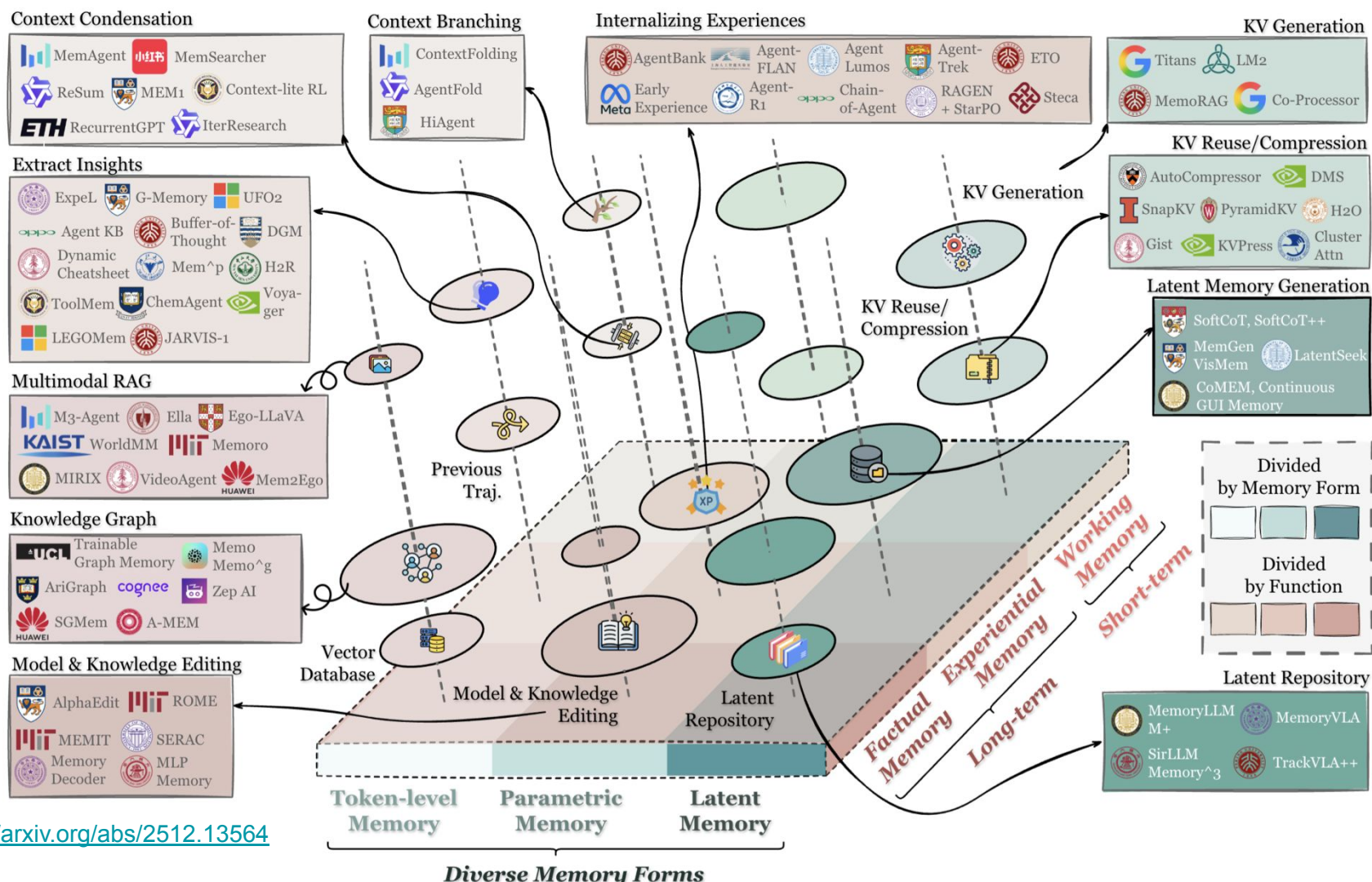


(c) Sparse Transformer (fixed)

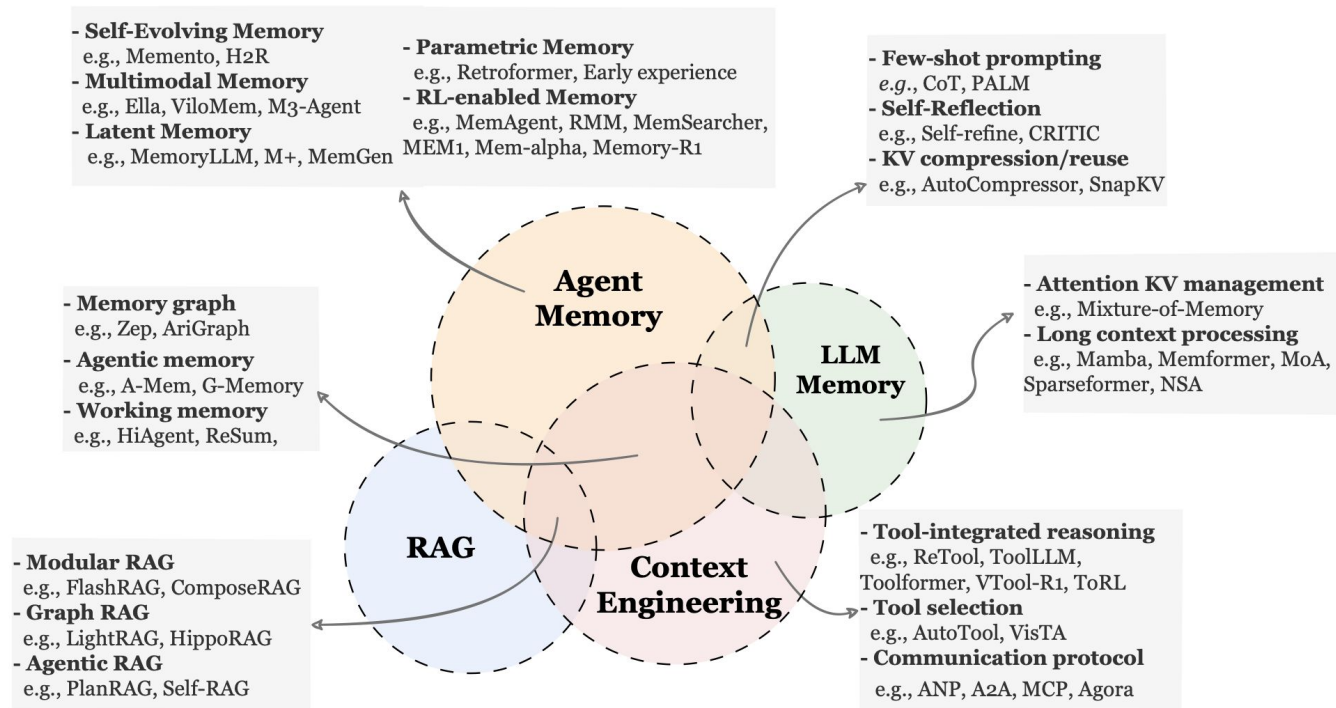
Efficient Attention



Memory in Transformers



Memory



Memory

Experience



e.g., ExpeL, AWM, ReasoningBank

Chunk



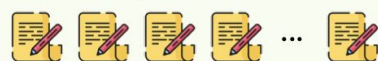
e.g., Nemori, Memo, MemOS

Dialogue



e.g., MemGPT, MemoryBank

Summary

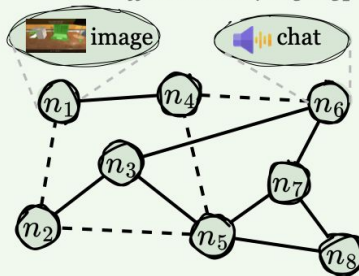


e.g., Think-in-Memory, RMM

(a) Flat Memory (1D)

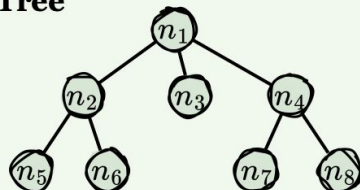
Graph

Memory graphs with different node/edge types



e.g., A-Mem, Memo[^]g, M3-Agent, D-SMART

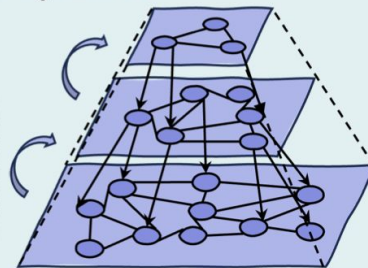
Tree



e.g., MemTree, TME, others

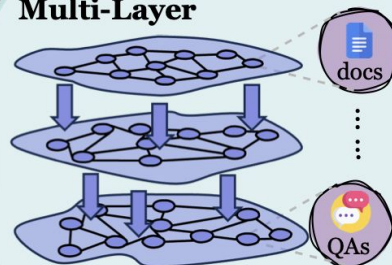
(b) Planar Memory (2D)

Pyramid



e.g., G-Memory, CAM, others

Multi-Layer

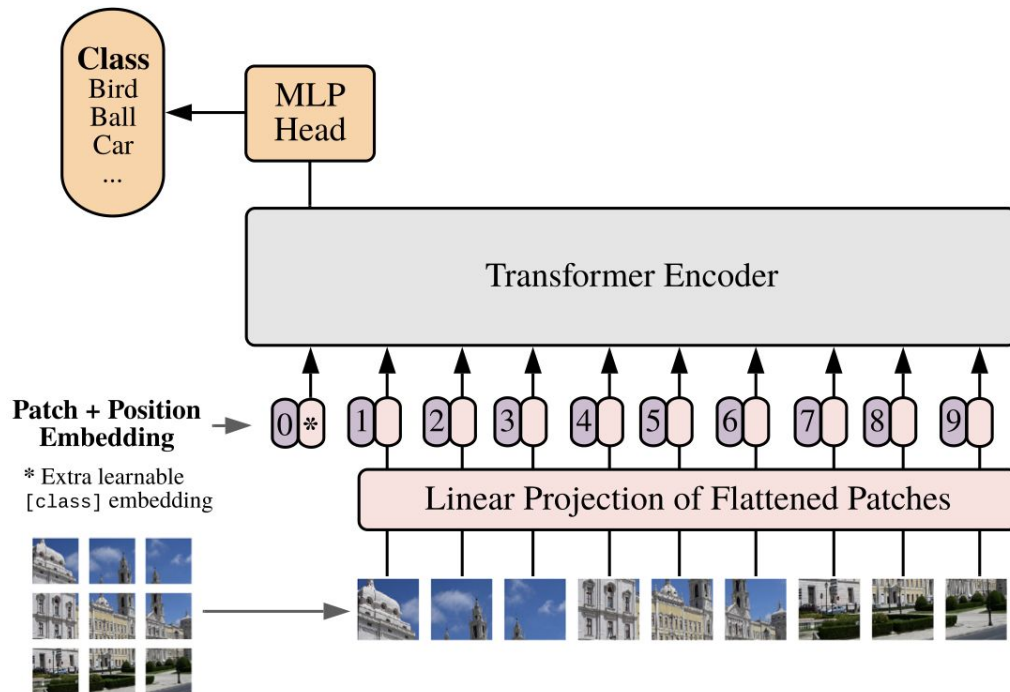


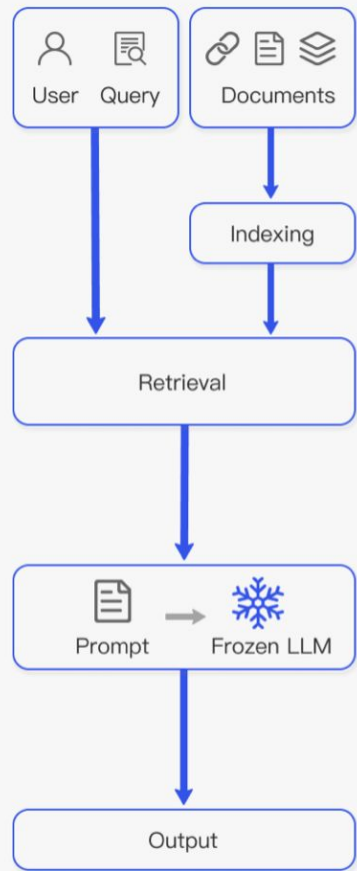
HiAgent, HippoRAG, SGMem

(c) Hierarchical (3D)

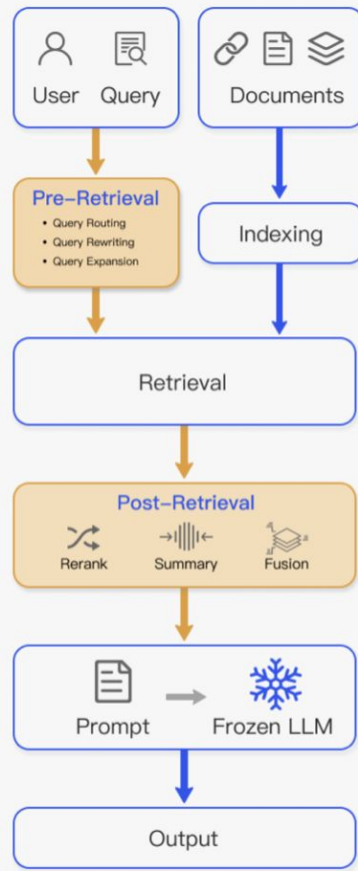
Transformer

Vision Transformer (ViT)



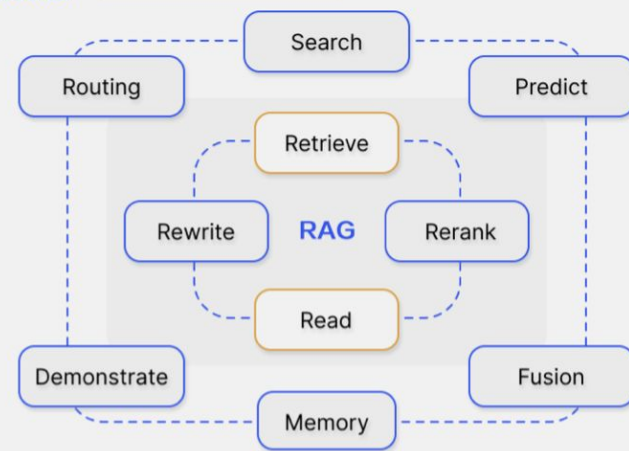


Naive RAG

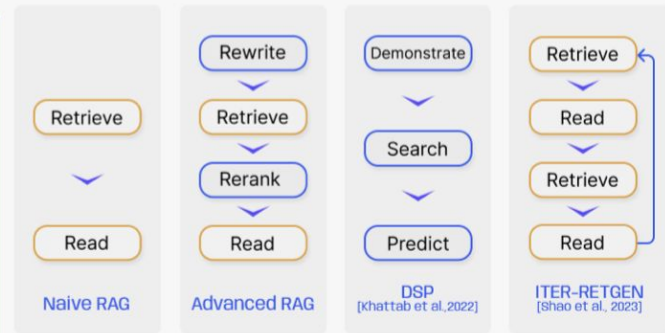


Advanced RAG

Modules

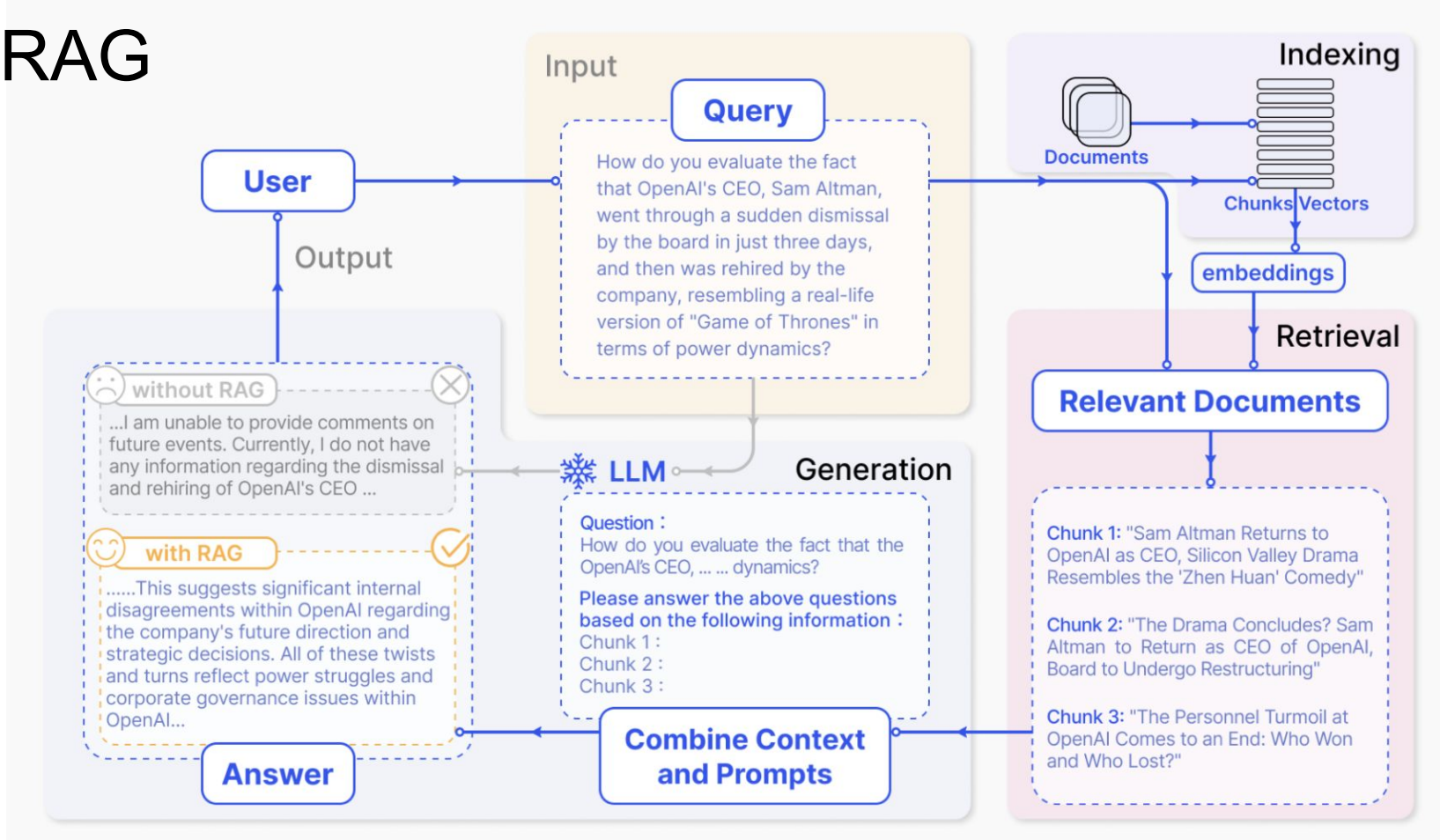


Patterns



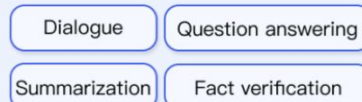
Modular RAG

RAG

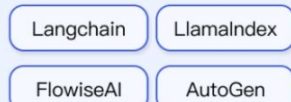


» RAG Ecosystem

Downstream Tasks



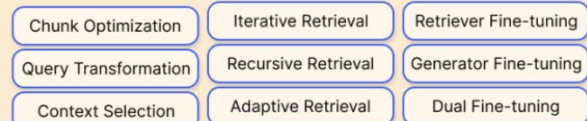
Technology Stacks



» The RAG Paradigm



» Techniques for Better RAG



» Key Issues of RAG

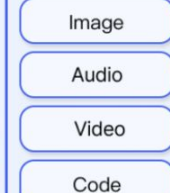


» RAG Prospect

Challenges



Modality Extension



Ecosystem



» Evaluation of RAG

Evaluation Target



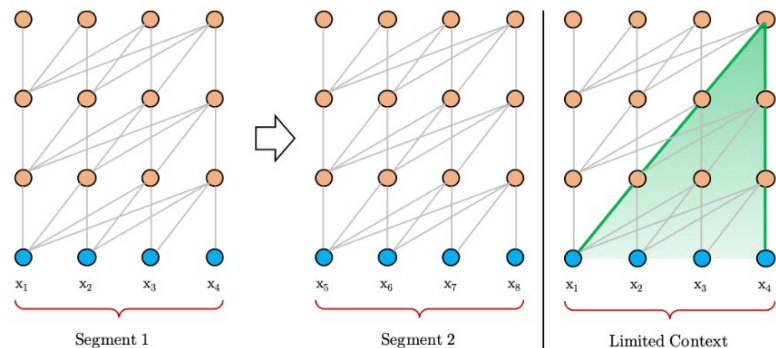
Evaluation Aspects



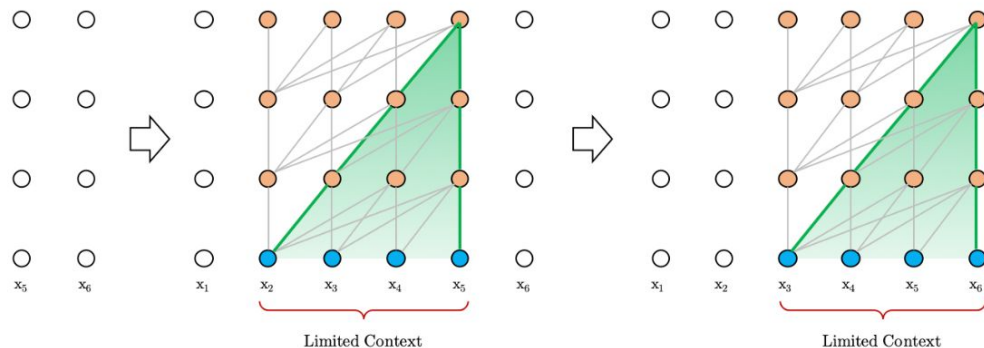
Evaluation Framework



Vanilla Transformer

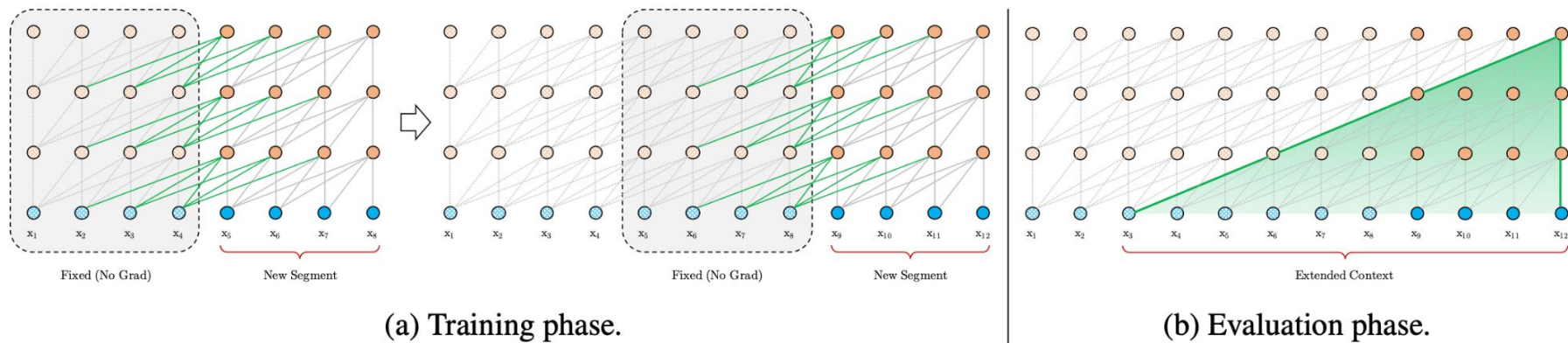


(a) Train phase.

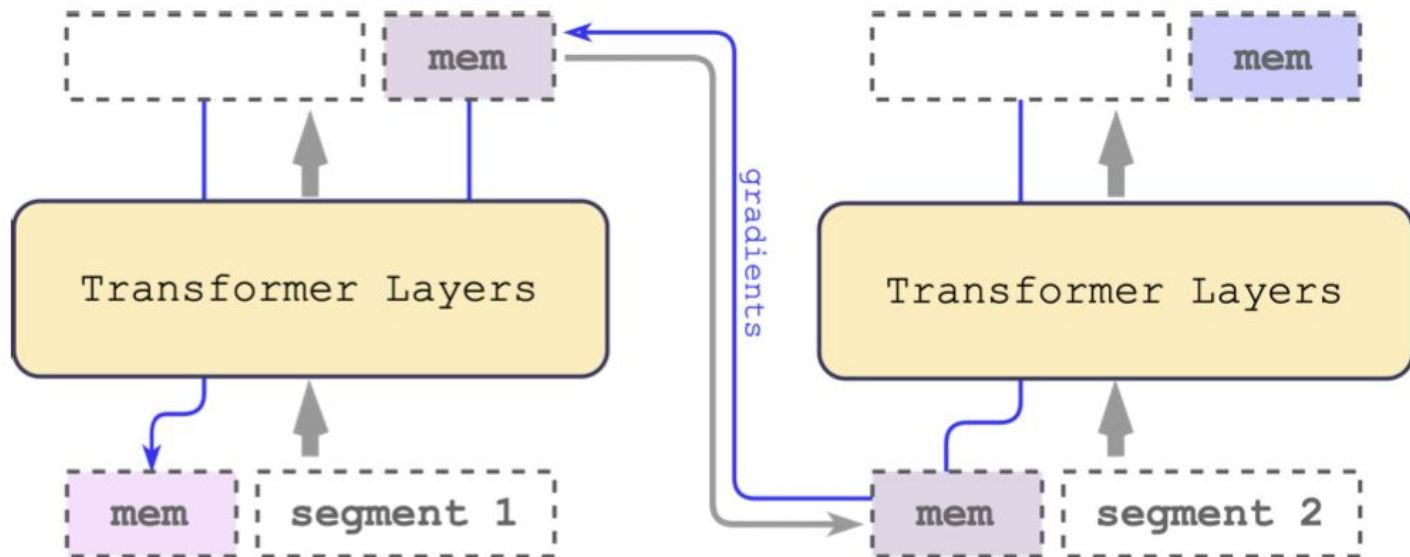


(b) Evaluation phase.

TransformerXL

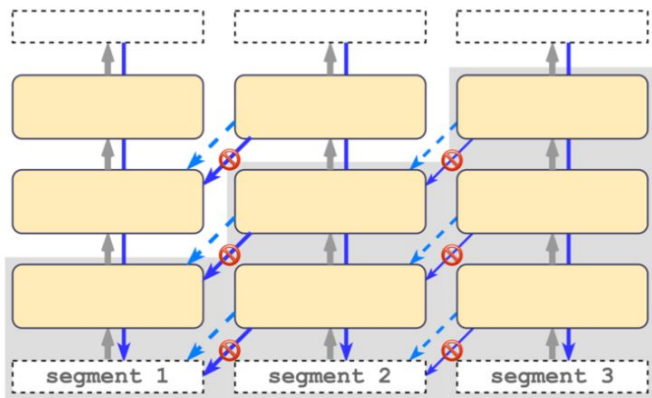


RMT

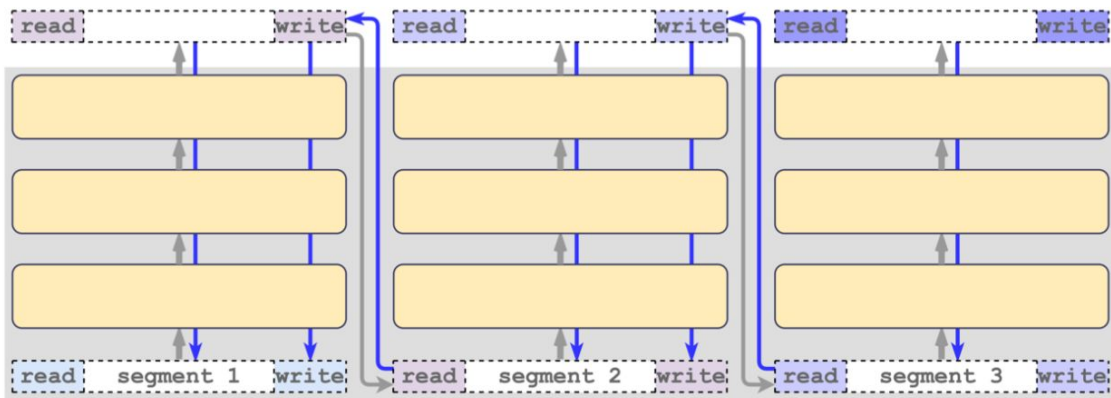


RMT

Transformer-XL



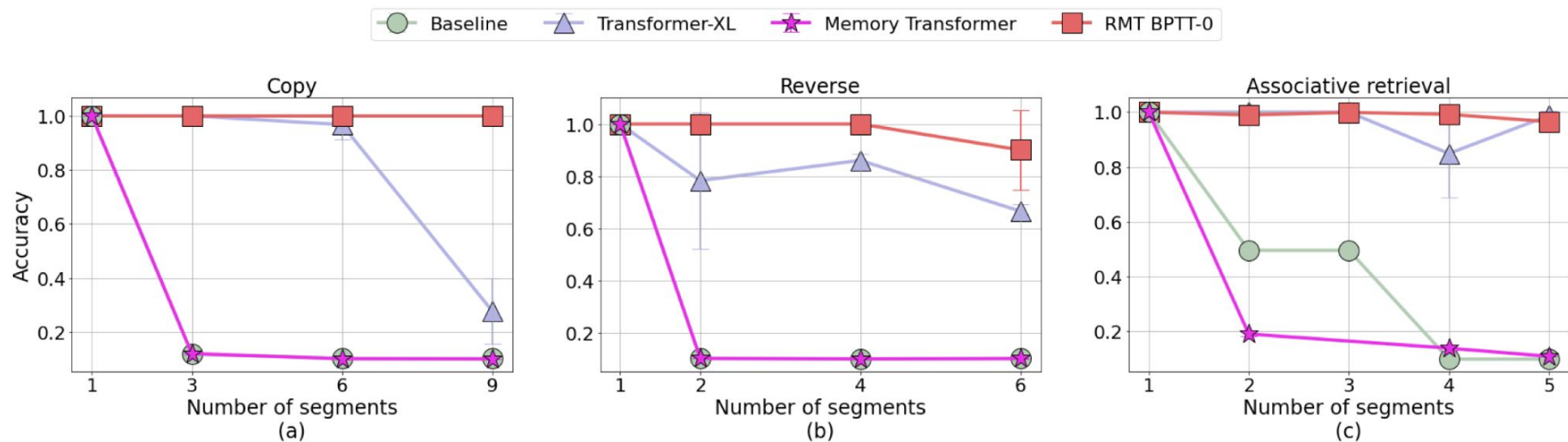
Recurrent Memory Transformer



Transformer layer  Attention to cached states 

Effective context  Forward pass  Gradients  Stop Gradient 

RMT



Memory Compression

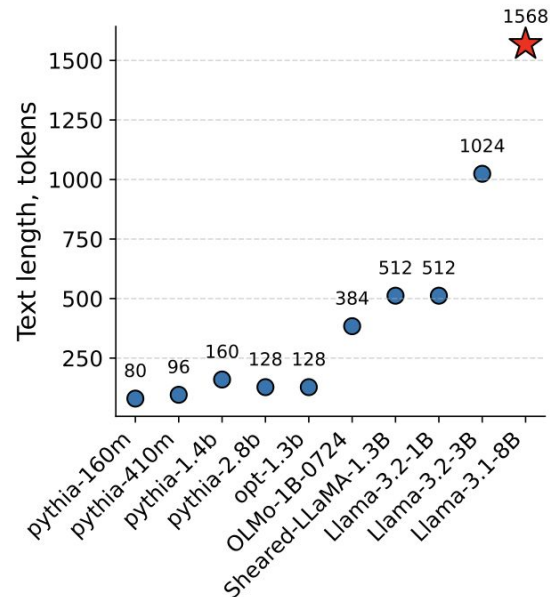
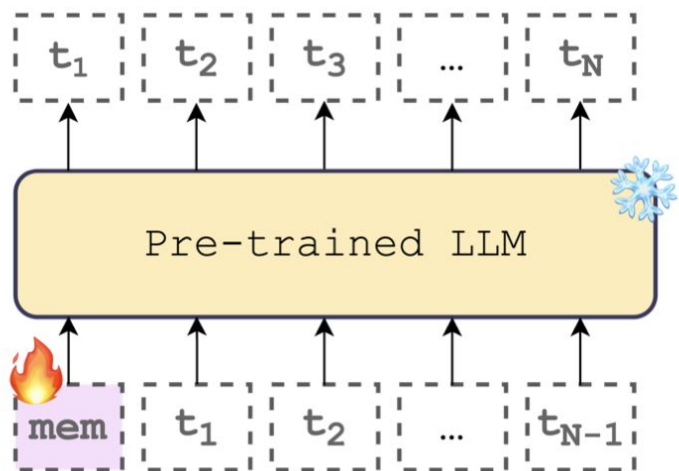
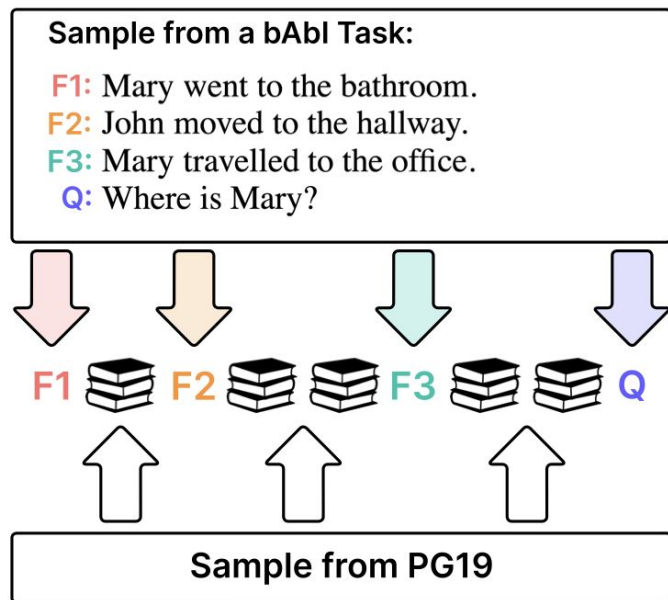


Figure 1: **How many tokens fit into a single input vector?** We estimate maximum number of tokens that can be decoded from a single input vector across various language models.

Memory Evaluation

a)

BABILong benchmark



b)

QA1: Single supporting fact

